



PHYS551: Advanced Monte Carlo Methods

Homework 6

Student:

Ömer Faruk AVCI

Instructor:

Prof. Dr. Taylan AKDOĞAN

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

May 17, 2026

Disclosures and References

AI Usage Disclosure: In accordance with the course policy, I would like to declare the extent of AI assistance used in this assignment:

- **Code & Visualizations:** AI tools were utilized to assist in debugging PyMC model definitions and to refine the output plots.
- **Document Preparation:** AI assistance was used to construct and format the LaTeX document.

Problem 1: Model 1 – Basquin’s Equation

Part (a): Prior Predictive Check

Basquin’s equation in log-space is:

$$\ln(N_i) = \alpha - \beta \ln(S_i), \quad N_i \sim \text{LogNormal}(\mu_i, \sigma)$$

I centered the log-stress by subtracting $\overline{\ln S}$ to reduce correlation between α and β during sampling. The priors I used:

- $\alpha \sim \mathcal{N}(13.5, 5)$ – around expected log-life at mean stress
- $\beta \sim \mathcal{N}(10, 5)$ – positive so that life decreases with stress
- $\sigma \sim \text{HalfNormal}(1)$

I sampled 500 prior predictive curves and plotted them on a log-log scale. Out of 500 draws, only 10 had $\beta < 0$ (about 2%), so the priors are reasonable.

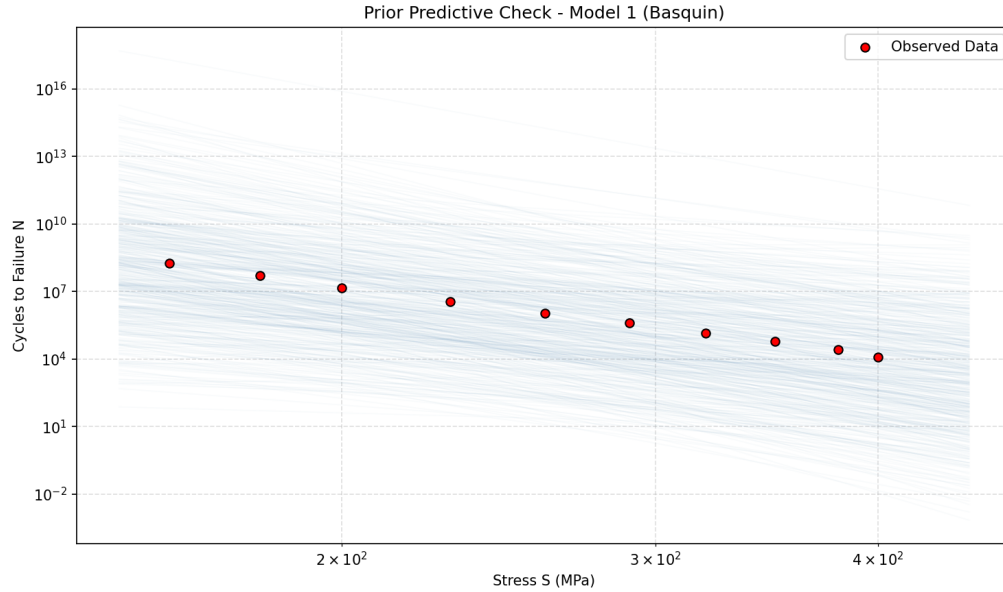


Figure 1: Prior predictive S-N curves (500 draws). Data shown in red.

Part (b): Inference & Diagnostics

Ran NUTS with 4 chains, 1000 tuning, 2000 draws each. Set `target_accept` = 0.95.

Model 1 – MCMC Summary

```
Parameter Mean SD R-hat Bulk ESS Tail ESS
-----
alpha 13.765 0.044 1.00 5161 4544
beta 10.224 0.142 1.00 5564 4150
sigma 0.134 0.042 1.00 4221 4235
Divergent transitions: 0
```

$\hat{R} = 1.00$ across the board, ESS values are all well above 400, and no divergences. The trace plots show good mixing across all 4 chains.

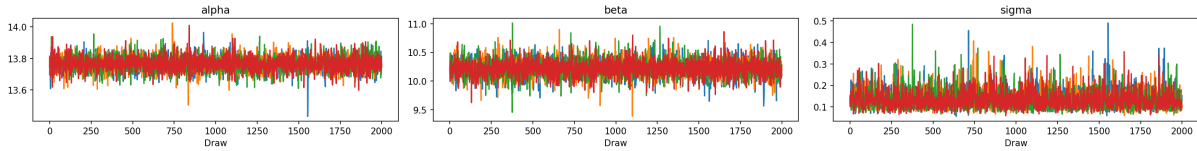


Figure 2: Trace plots for Model 1. All chains overlap well.

Part (c): Posterior Predictive Check

Generated the posterior predictive distribution by propagating the posterior samples through the model (including the σ noise). Computed the 95% credible interval from the 2.5th and 97.5th percentiles.

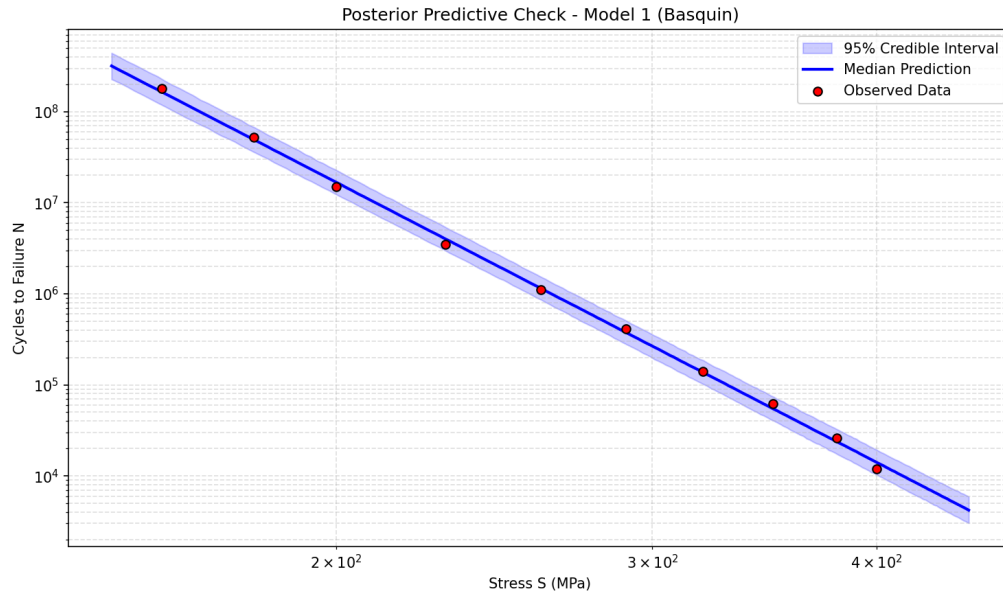


Figure 3: Posterior predictive check. All data points fall inside the 95% band.

All 10 data points sit comfortably within the credible band and the median line tracks them closely. The band is fairly narrow ($\sigma \approx 0.13$), which tells us Basquin's power law is a pretty good fit for this data.

Problem 2: Model 2 – Stromeyer’s Equation

Part (a) & (b): Model and Inference

Stromeyer adds a fatigue limit S_e :

$$\mu_i = \alpha - \beta \ln(S_i - S_e)$$

I used $S_e \sim \text{Uniform}(0, 155)$ so that $S_i - S_e > 0$ for all data points (lowest stress is 160 MPa). The priors for α and β are similar to Model 1, though I widened α to $\mathcal{N}(40, 15)$ since the Stromeyer parameterization shifts the intercept higher.

This model has a tricky posterior geometry – when S_e gets close to the lowest stress value, tiny changes in S_e cause huge swings in $\ln(S - S_e)$, creating a funnel shape. To deal with this I set `target_accept = 0.99` and doubled the tuning steps to 2000. All 4 chains hit the maximum tree depth warning, which is expected for this kind of geometry, but the sampler still converged with 0 divergences.

Model 2 – MCMC Summary

```
Parameter Mean SD R-hat Bulk ESS Tail ESS
-----
alpha 62.7 4.6 1.00 977 1390
beta 8.96 0.72 1.00 982 1411
sigma 0.144 0.052 1.00 1619 1741
Se 30.0 16.7 1.00 979 1217
Divergent transitions: 0
```

The ESS values are noticeably lower than Model 1 (around 1000 vs 5000), which reflects the harder posterior geometry. Still above the minimum threshold though.

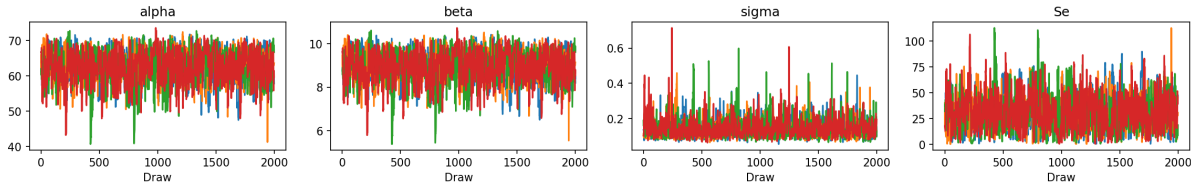


Figure 4: Trace plots for Model 2. S_e explores a wide range but chains still mix.

Part (c): Fatigue Limit Results

Fatigue Limit Estimate

```
Se (Fatigue Limit):
Mean = 30.0 MPa
94
```

The estimated fatigue limit is $S_e = 30.0 \pm 16.7$ MPa. The 94% HDI is very wide and almost touches zero, meaning the data doesn’t really constrain S_e well. This makes sense physically – our tested stresses (160–400 MPa) are all way above any reasonable endurance limit for this alloy, so the data just doesn’t have enough information to pin down where the material would stop failing.

Problem 3: Model Comparison (PSIS-LOO)

Part (a): LOO Cross-Validation

I used `az.compare()` to run PSIS-LOO on both models.

LOO Comparison

```
rank  ELPD  p_eff  ELPD.diff  weight  SE  dSE  warning
-----
Basquin  0 -130.0  2.2  0.0  1.0  9.6  0.0  False
Stromeyer 1 -130.0  3.6 -2.0  0.0  8.9  2.1  True
```

One thing worth noting: the Stromeyer model triggered a Pareto $\hat{k} > 0.7$ warning during LOO computation, which means that for at least one data point, the importance sampling approximation is unreliable. This happens when removing a single observation changes the posterior significantly – basically one of the data points is very influential for the Stromeyer model. The Basquin model had no such issue.

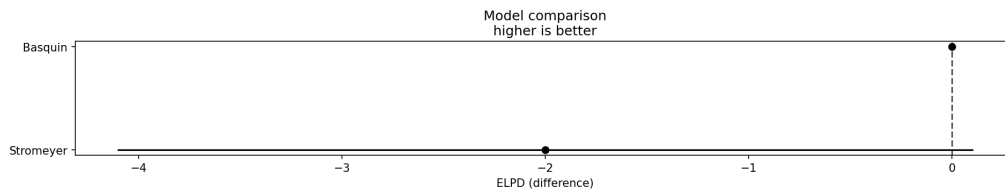


Figure 5: ELPD comparison. Higher is better. Error bars show standard error.

Part (b): Conclusion

The ELPD difference between the two models is $\Delta\text{ELPD} = 2.0$ with a standard error of 2.1. Since the difference is smaller than the standard error, neither model is clearly better at predicting unseen data. However, there are several reasons to prefer the simpler Basquin model:

First, Basquin uses fewer effective parameters ($p_{\text{eff}} = 2.2$ vs 3.6) and Occam's razor favors parsimony when predictive power is equivalent. Second, the Stromeyer model triggers the Pareto $\hat{k}$ diagnostic warning, indicating its LOO estimate is less trustworthy. Third, the fatigue limit S_e has such a wide posterior (HDI nearly touching zero) that the data doesn't support the existence of an endurance limit in this stress range. The added complexity of Stromeyer's equation is not justified here, and Basquin's power law remains the better engineering model for this dataset.

Appendix: Source Code

hw6.py – Complete Implementation

```
1 """
2 # PHYS 551: Advanced Monte Carlo Methods
3 # Homework 6 - Source Code
4 # Student: Omer Faruk Avci
5 # Instructor: Prof. Dr. Taylan Akdogan
6 """
7 import pymc as pm
8 import numpy as np
9 import matplotlib.pyplot as plt
10 import arviz as az
11
12 # =====
13 # DATA
14 # =====
15 stress = np.array([400, 380, 350, 320, 290, 260, 230, 200, 180, 160], dtype=float)
16 cycles = np.array([1.2e4, 2.6e4, 6.2e4, 1.4e5, 4.1e5, 1.1e6, 3.5e6, 1.5e7, 5.2e7, 1.8e8])
17
18 # Centering trick to reduce alpha-beta correlation in sampling
19 log_S = np.log(stress)
20 log_S_mean = np.mean(log_S)
21 log_S_centered = log_S - log_S_mean
22
23 # Smooth stress range for plotting
24 s_plot = np.linspace(150, 450, 200)
25 log_s_plot_c = np.log(s_plot) - log_S_mean
26
27 # =====
28 # PROBLEM 1: MODEL 1 - BASQUIN'S EQUATION (50 pts)
29 #  $N = A * S^{-beta} \Rightarrow \ln(N) = \alpha - beta * \ln(S)$ 
30 # =====
31
32 with pm.Model() as model1:
33     # Weakly informative priors
34     alpha = pm.Normal('alpha', mu=13.5, sigma=5)
35     beta = pm.Normal('beta', mu=10, sigma=5)
36     sigma = pm.HalfNormal('sigma', sigma=1)
37
38     mu_i = alpha - beta * log_S_centered
39     N_obs = pm.LogNormal('N_obs', mu=mu_i, sigma=sigma, observed=cycles)
40
41 # --- 1a: Prior Predictive Check ---
42
43 with model1:
44     prior = pm.sample_prior_predictive(draws=500, random_seed=42)
45
46 pr_alpha = prior.prior['alpha'].values.flatten()
47 pr_beta = prior.prior['beta'].values.flatten()
48
49 plt.figure(figsize=(10, 6))
50 for i in range(len(pr_alpha)):
51     mu_line = pr_alpha[i] - pr_beta[i] * log_s_plot_c
52     plt.plot(s_plot, np.exp(mu_line), color='steelblue', alpha=0.03, lw=0.7)
53
54 plt.scatter(stress, cycles, color='red', edgecolor='black', zorder=5,
55             label='Observed Data')
56 plt.yscale('log'); plt.xscale('log')
57 plt.xlabel('Stress S (MPa)')
58 plt.ylabel('Cycles to Failure N')
59 plt.title('Prior Predictive Check - Model 1 (Basquin)')
60 plt.legend(); plt.grid(True, which='both', ls='--', alpha=0.4)
61 plt.tight_layout()
62 plt.savefig('prior_predictive_m1.png', dpi=150)
63 plt.close()
64
65 n_neg = np.sum(pr_beta < 0)
66 print(f"Prior draws with beta < 0 (unphysical): {n_neg}/{len(pr_beta)}")
67
68 # --- 1b: Inference & Diagnostics ---
69
70 with model1:
```

```

72     trace_m1 = pm.sample(draws=2000, tune=1000, chains=4,
73                           target_accept=0.95, random_seed=42)
74     pm.compute_log_likelihood(trace_m1)
75
76     summary1 = az.summary(trace_m1, var_names=['alpha', 'beta', 'sigma'])
77     print("\n=== Model 1 Summary ===")
78     print(summary1)
79
80     divs1 = trace_m1.sample_stats['diverging'].values.sum()
81     print(f"Divergent transitions: {divs1}")
82
83     az.plot_trace(trace_m1, var_names=['alpha', 'beta', 'sigma'])
84     plt.tight_layout()
85     plt.savefig('trace_m1.png', dpi=150)
86     plt.close()
87
88     # --- 1c: Posterior Predictive Check ---
89
90     post_alpha = trace_m1.posterior['alpha'].values.flatten()
91     post_beta = trace_m1.posterior['beta'].values.flatten()
92     post_sigma = trace_m1.posterior['sigma'].values.flatten()
93
94     rng = np.random.default_rng(42)
95     mu_pred = post_alpha[:, None] - post_beta[:, None] * log_s_plot_c
96     log_N_pred = mu_pred + rng.normal(0, post_sigma[:, None], size=mu_pred.shape)
97     N_pred = np.exp(log_N_pred)
98
99     low = np.percentile(N_pred, 2.5, axis=0)
100    high = np.percentile(N_pred, 97.5, axis=0)
101    med = np.percentile(N_pred, 50, axis=0)
102
103    plt.figure(figsize=(10, 6))
104    plt.fill_between(s_plot, low, high, color='blue', alpha=0.2,
105                    label='95% Credible Interval')
106    plt.plot(s_plot, med, 'b-', lw=2, label='Median Prediction')
107    plt.scatter(stress, cycles, color='red', edgecolor='black', zorder=5,
108               label='Observed Data')
109    plt.yscale('log'); plt.xscale('log')
110    plt.xlabel('Stress S (MPa)')
111    plt.ylabel('Cycles to Failure N')
112    plt.title('Posterior Predictive Check - Model 1 (Basquin)')
113    plt.legend(); plt.grid(True, which='both', ls='--', alpha=0.4)
114    plt.tight_layout()
115    plt.savefig('ppc_m1.png', dpi=150)
116    plt.close()
117
118
119    # =====
120    # PROBLEM 2: MODEL 2 - STROMEYER'S EQUATION (30 pts)
121    #  $N = A * (S - Se)^{-beta} \Rightarrow \mu = \alpha - beta * \ln(S - Se)$ 
122    # =====
123
124    with pm.Model() as model2:
125        alpha2 = pm.Normal('alpha', mu=40, sigma=15)
126        beta2 = pm.Normal('beta', mu=8, sigma=5)
127        sigma2 = pm.HalfNormal('sigma', sigma=1)
128
129        # Fatigue limit: bounded so  $S - Se > 0$  for all data points
130        Se = pm.Uniform('Se', lower=0, upper=155)
131
132        # Stromeier model (pm.math.log because Se is a random variable)
133        mu_i2 = alpha2 - beta2 * pm.math.log(stress - Se)
134        N_obs2 = pm.LogNormal('N_obs', mu=mu_i2, sigma=sigma2, observed=cycles)
135
136        # Higher target_accept + more tuning to handle funnel geometry
137        trace_m2 = pm.sample(draws=2000, tune=2000, chains=4,
138                              target_accept=0.99, random_seed=42)
139        pm.compute_log_likelihood(trace_m2)
140
141    summary2 = az.summary(trace_m2, var_names=['alpha', 'beta', 'sigma', 'Se'])
142    print("\n=== Model 2 Summary ===")
143    print(summary2)
144
145    divs2 = trace_m2.sample_stats['diverging'].values.sum()
146    print(f"Divergent transitions: {divs2}")
147
148    # --- 2c: Se results (94% HDI) ---

```

```

149 se_samples = trace_m2.posterior['Se'].values.flatten()
150 se_hdi = az.hdi(trace_m2, var_names=['Se'], prob=0.94)
151
152 print(f"\nFatigue Limit Se:")
153 print(f"  Mean = {np.mean(se_samples):.1f} MPa")
154 print(f"  94% HDI = [{se_hdi['Se'].values[0]:.1f}, "
155         f"{se_hdi['Se'].values[1]:.1f}] MPa")
156
157 az.plot_trace(trace_m2, var_names=['alpha', 'beta', 'sigma', 'Se'])
158 plt.tight_layout()
159 plt.savefig('trace_m2.png', dpi=150)
160 plt.close()
161
162
163 # =====
164 # PROBLEM 3: MODEL COMPARISON - PSIS-LOO (20 pts)
165 # =====
166
167 comparison = az.compare({"Basquin": trace_m1, "Stromeyer": trace_m2})
168 print("\n=== LOO Model Comparison ===")
169 print(comparison)
170
171 az.plot_compare(comparison)
172 plt.tight_layout()
173 plt.savefig('loo_comparison.png', dpi=150)
174 plt.close()
175
176 print("\nAll figures saved. Done.")

```